Below is a complete C++ program that implements a **Music Playlist** using a **Doubly Linked List**.

Features

- Add a song to the end of the playlist
- Remove a song by title
- Display playlist in forward order
- Display playlist in reverse order
- Menu-driven program

```
#include <iostream>
#include <string>
using namespace std;

// Node structure
class Node {
public:
    string song;
    Node* next;
    Node* prev;

    Node(string title) {
        song = title;
        next = nullptr;
        prev = nullptr;
    }
};

// Doubly Linked List class
class Playlist {
private:
    Node* head;
    Node* tail;

public:
    Playlist() {
        head = nullptr;
        tail = nullptr;
    }
```

```
// Add a song
    void addSong(string title) {
        Node* newNode = new Node(title);

        if (head == nullptr) {
            head = tail = newNode;
        } else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }

        cout << "Song added successfully.\n";
    }

   // Remove a song
   void removeSong(string title) {
if (head == nullptr) { cout << "Playlist is empty.\n";
        return; }

        Node* temp = head;

        while (temp != nullptr && temp->song != title) {
           temp = temp->next;
        }

        if (temp == nullptr) {
            cout << "Song not found.\n";
            return;
        }

        // If deleting the head
        if (temp == head) {
            head = head->next;
            if (head != nullptr)
                head->prev = nullptr;
            else
                tail = nullptr;
        }
```

```
        // If deleting the tail
        else if (temp == tail) {
            tail = tail->prev;
            tail->next = nullptr;
        }
        // Deleting a middle node
        else {
            temp->prev->next = temp->next;
            temp->next->prev = temp->prev;
        }

        delete temp;
        cout << "Song removed successfully.\n";
    }

    // Display playlist forward
    void displayForward() {
        if (head == nullptr) {
            cout << "Playlist is empty.\n";
            return;
        }

        cout << "\nPlaylist (Forward):\n";
        Node* temp = head;

        while (temp != nullptr) {
            cout << temp->song << endl;
            temp = temp->next;
        }
    }

    // Display playlist reverse
    void displayReverse() {
        if (tail == nullptr) {
            cout << "Playlist is empty.\n"; return;
        }

        cout << "\nPlaylist (Reverse):\n";
        Node* temp = tail;
        while (temp != nullptr) {
            cout << temp->song << endl;
            temp = temp->prev;
        }
    }
};
```

```
int main() {
    Playlist playlist;
    int choice;
    string title;

    do {
        cout << "\n===== Music Playlist Menu =====\n";
        cout << "1. Add Song\n";
        cout << "2. Remove Song\n";
        cout << "3. Display Playlist (Forward)\n";
        cout << "4. Display Playlist (Reverse)\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;cin.ignore();
        switch (choice) {
        case 1:
            cout << "Enter song title: ";
            getline(cin, title);
            playlist.addSong(title); break;

        case 2:
            cout << "Enter song title to remove: ";
            getline(cin, title);
            playlist.removeSong(title);  break;

        case 3:
            playlist.displayForward();
            break;

        case 4:
            playlist.displayReverse();    break;

        case 5:
            cout << "Exiting program...\n";  break;

        default:
            cout << "Invalid choice.\n";
        }

    } while (choice != 5);

    return 0;
}
```

Sample Output

```
===== Music Playlist Menu =====
1. Add Song
2. Remove Song
3. Display Playlist (Forward)
4. Display Playlist (Reverse)
5. Exit
Enter your choice: 1
```

Time Complexity

Operation	Time Complexity
Add Song	**O(1)**
Remove Song	**O(n)**
Display Forward	**O(n)**
Display Reverse	**O(n)**